Micro Services

- Day 3

Monitoring Spring Boot Applications

Resilience & fault tolerance

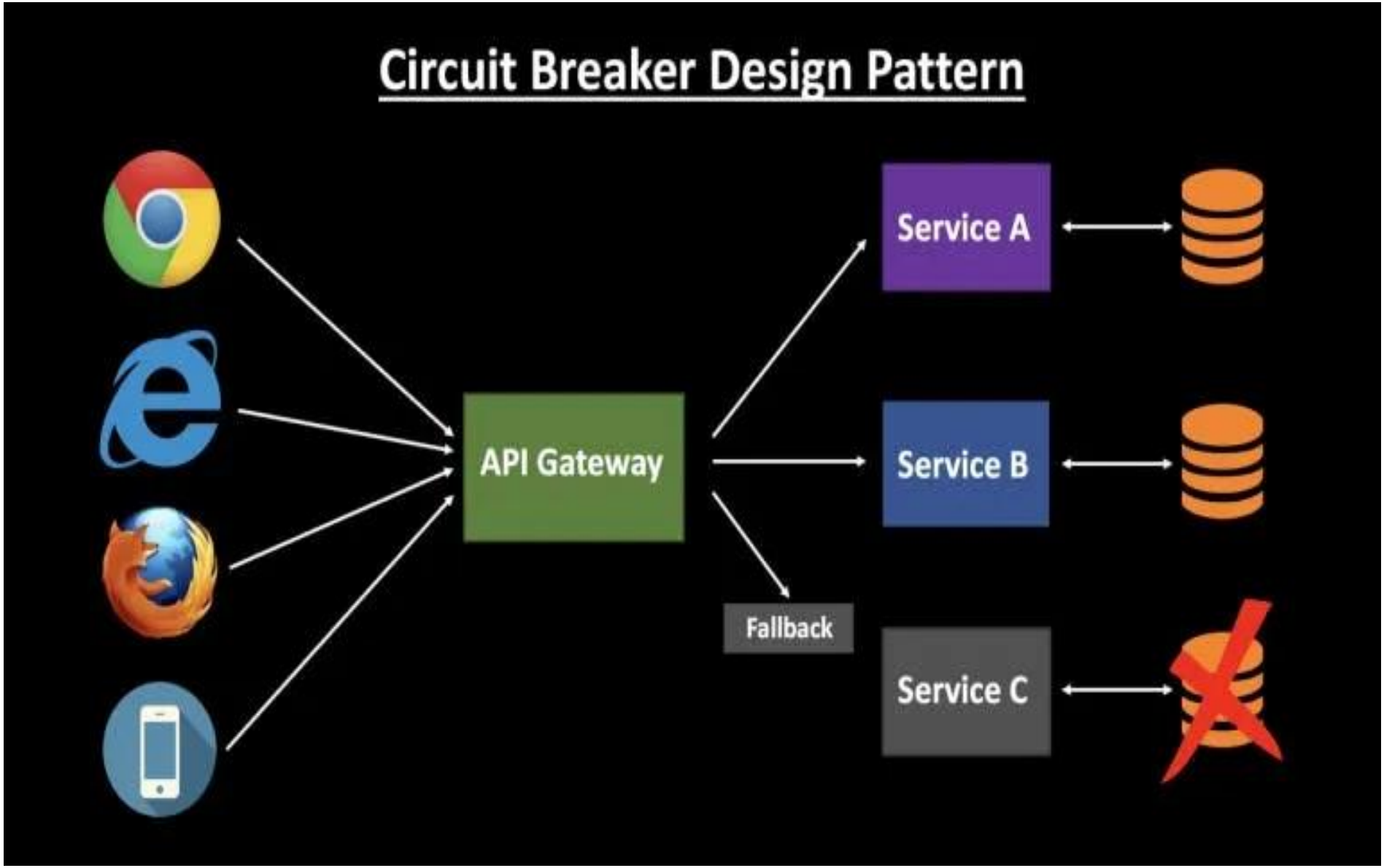
- **Fault tolerance** is *"Tolerance the system has for a fault"* means how much tolerant microservice application is or what is the impact when there is a fault.

✓ will the whole application go down or part of functionality go down or there are some arrangements to handle failures.

✓ **Fault tolerance** and **resilience** in microservices ensure systems remain functional despite component failures or network issues, preventing cascading outages.

- **Resilience** means how many faults the system can tolerate.
  Key strategies include -
  a) isolating services,
  b) implementing <u>circuit breakers</u> to stop calls to failing components,
  c) using <u>retries</u> for transient errors, and setting strict timeouts.

✓ These techniques maintain system availability and gracefully degrade functionality rather than crashing completely.


Circuit Breaker Design Pattern
API Gateway
Service A
Service B
Service C
Fallback

Key techniques for Resilient Microservices

- **Circuit Breaker Pattern:** Detects failures and prevents the system from constantly trying to call a failing service, allowing it to recover.
- **Bulkhead Pattern :** Partitions system resources (e.g., thread pools) into isolated pools so that if one service fails, others continue to function.
- **Retry Pattern:** Automatically retries a failed request, useful for transient network glitches, but should be used with limits to avoid overloading services.
- **Timeout Pattern:** Stops waiting for a response after a set time, preventing a slow service from cascading delays throughout the system.
- **Fallback Methods:** Provides a default response or alternative functionality when a service call fails.
- **Service Isolation:** Ensuring services are independent prevents a failure in one service (e.g., ordering) from crashing another (e.g., user profile)

contd..

- When building microservices, it is crucial to design with the expectation that failures will occur. This proactive approach involves planning and building systems that can gracefully handle failures and recover with minimal impact.
- **Self-Healing Mechanisms** – Self-healing mechanisms are another critical component of fault tolerance. These mechanisms allow services to recover from failures without external intervention.

Observability & Monitoring

- Observability and monitoring in microservices are essential, complementary practices for managing distributed system health.
- **Monitoring** tracks predefined metrics (e.g., CPU, latency) to tell admin *if* a system is failing.
- **Observability** uses logs, metrics, and traces to explain *why* it is failing by providing deep insights into internal states across complex service dependencies.

Key Metrics to Monitor

- Basic Metrics: CPU usage, memory usage, disk space.
- Application-Specific Metrics: HTTP requests, database queries, cache hits/misses.
- Health Checks: Application health indicators (/actuator/health).

Using Actuator for Monitoring

- Overview: Built-in support for monitoring and managing Spring Boot applications.
- Endpoints: /actuator endpoints for metrics, health checks, info, etc.

1. Health Monitoring: Use the /actuator/health endpoint to check the health status.
2. Metrics Monitoring: Access /actuator/metrics for app metrics like memory usage, CPU usage, etc.
3. Application Info: Provide application-specific information using /actuator/info.

4. HTTP Tracing: Monitor HTTP request and response traces with /actuator/httptrace.

Enabling Actuator in Spring Boot

Add dependency to pom.xml file

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Configure Actuator in application.properties:

```
spring.application.name=violetApple
management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always
```

Exposing Actuator Endpoints O/P

Shows overall status of your application

- Check Health:
  - Navigate to http://localh

diskSpace: Information about disk space availability.

ping: Basic connectivity check

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize JSON


```
1  {
2      "status": "UP",
3      "components": {
4          "diskSpace": {
5              "status": "UP",
6              "details": {
7                  "total": 498815614976,
8                  "free": 401035259904,
9                  "threshold": 10485760,
10                 "path": "C:\\violetApple\\.",
11                 "exists": true
12             }
13         },
14         "ping": {
15             "status": "UP"
16         }
17     }
18 }
```

Introduction to Spring Boot Logging

- Logging is the process of recording information about a program's execution, which helps in debugging and monitoring.
- Configures Logback as the default logging framework.
- Includes Logback and SLF4J dependencies by default.

Configuring Logging in Spring Boot

- Logging settings can be configured in application.properties
- Each log level indicates the severity of the messages: ERROR (most severe), WARN, INFO, DEBUG, TRACE (most detailed)

```
<!-- Spring Boot Starter for logging -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-logging</artifactId>
</dependency>
```

Adding Spring boot Logging dependency

Example configuration in the application.properties

```
# Logging configuration
logging.level.org.springframework=INFO
logging.level.com.myapp=DEBUG
```

Example usage of logging

Depending upon the severity, use the appropriate logging level

```
@GetMapping("/log")  no usages
public String logExample() {
    // Log msg at different levels
    logger.trace("This is a TRACE level message");
    logger.debug("This is a DEBUG level message");
    logger.info("This is an INFO level message");
    logger.warn("This is a WARN level message");
    logger.error("This is an ERROR level message");
}
```

Introduction to Spring Boot Admin

- An open-source project designed to help manage and monitor Spring Boot applications
- Keep track of various aspects of applications in real time.
- Managing Application Properties and Environment.

Setting Up Spring Boot Admin

Add Dependency: Include the spring-boot-admin-starter-server dependency in your Spring Boot application's pom.xml

```
spring.boot.admin.client.url=http://localhost:8080/
```

Add to application.properties

Add the @EnableAdminServer annotation to your main application class to enable admin server features.

```
@SpringBootApplication
@EnableAdminServer
@EnableWebMvc
public class VioletAppleApplication {
    public static void main(String[] args) {
        SpringApplication.run(VioletAppleApplication.cla
    }
```

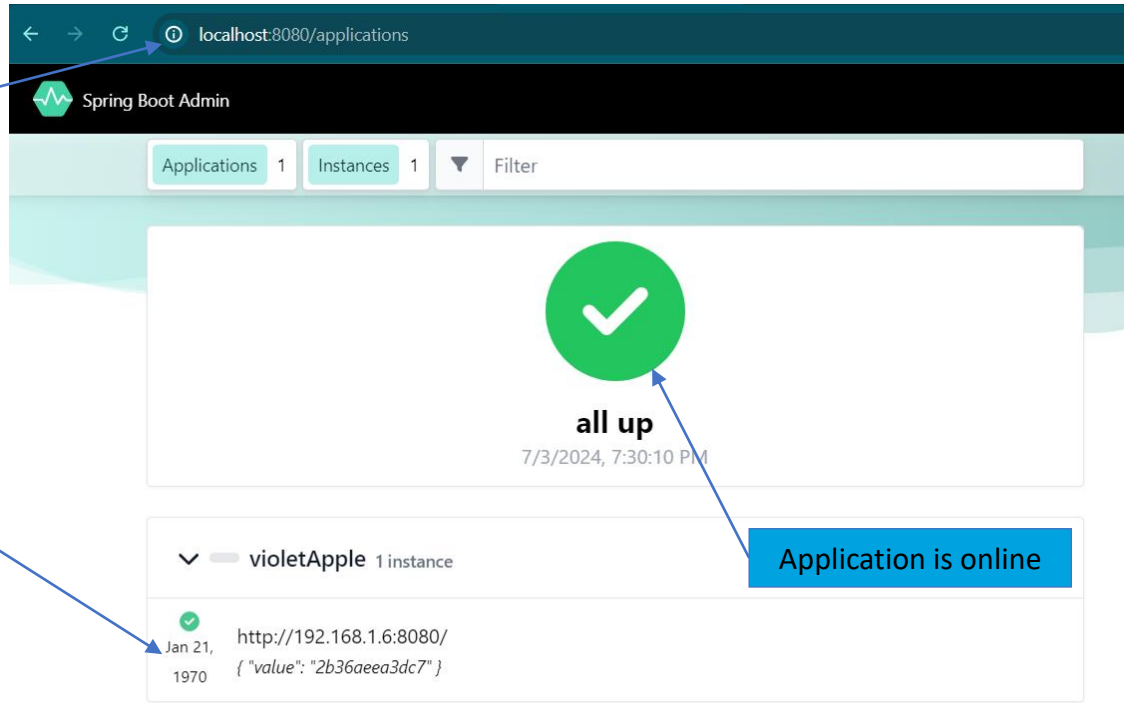
```
<!-- https://mvnrepository.com/artifact/de.codecentric/spring-b
<dependency>
    <groupId>de.codecentric</groupId>
    <artifactId>spring-boot-admin-starter-client</artifactId>
    <version>3.3.2</version>
</dependency>
```

```
<!-- https://mvnrepository.com/artifact/de.codecentric/spring-b
<dependency>
    <groupId>de.codecentric</groupId>
    <artifactId>spring-boot-admin-starter-server</artifactId>
    <version>3.3.2</version>
</dependency>
```

Monitoring and Managing

By default runs on localhost:8080

Registered application will be seen



Application is online

Observability tools

- **What are observability tools?**
  - ✓ Observability tools are systems that help you understand what's happening inside your applications and infrastructure. They do this by collecting and analyzing telemetry, specifically **logs**, **metrics**, and **traces**. These are often called the three pillars of observability:
  - ✓ **Logs** are timestamped records of events, including errors, warnings, requests, and system outputs.
  - ✓ **Metrics** are numeric measurements taken over time, such as CPU usage, memory usage, or request rates.
  - ✓ **Traces** follow individual requests as they move through different services or systems.
  - ✓ By examining these three pillars of data, you can get a comprehensive view of what your system is doing, how it's performing, and most importantly, where it's struggling or failing.

Zipkin

- Zipkin is a widely used open-source observability tool, specifically focused on **distributed tracing**.
- It helps developers troubleshoot latency issues and understand request flows in complex microservice architectures.
- Here is why it is classified as an observability tool:
  - ✓ **Distributed Tracing:** It acts as a GPS for requests, mapping how they travel through different services and identifying bottlenecks.
  - ✓ **Timing Data Collection:** It captures how long service calls take, which is critical for performance analysis.
  - ✓ **Visualization:** It provides a web interface to visualize the lifecycle of a request, helping pinpoint which service in a chain is causing errors or delays.
  - ✓ **Core Component:** While it primarily handles one of the three pillars of observability (traces), it is often integrated with logs and metrics tools for full system visibility.

Starting with Zipkin

- Zipkin is an open-source distributed tracing system that helps gather timing data
- Distributed tracing allows tracking the entire journey of a request, identifying bottlenecks

Setting Up Zipkin Server

- Download Zipkin server from https://zipkin.io/pages/quickstart

Click on latest release to download

Java

If you have Java 17 or higher installed,
latest release as a self-contained execu

```
PS C:\> java -jar .\zipkin-server-3.4.0-exec.jar
```

- Run the zipkin server by running following in power shell.

Setting Up Zipkin Server

```
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-tracing-bridge-brave</artifactId>
</dependency>
<dependency>
    <groupId>io.zipkin.reporter2</groupId>
    <artifactId>zipkin-reporter-brave</artifactId>
</dependency>
```

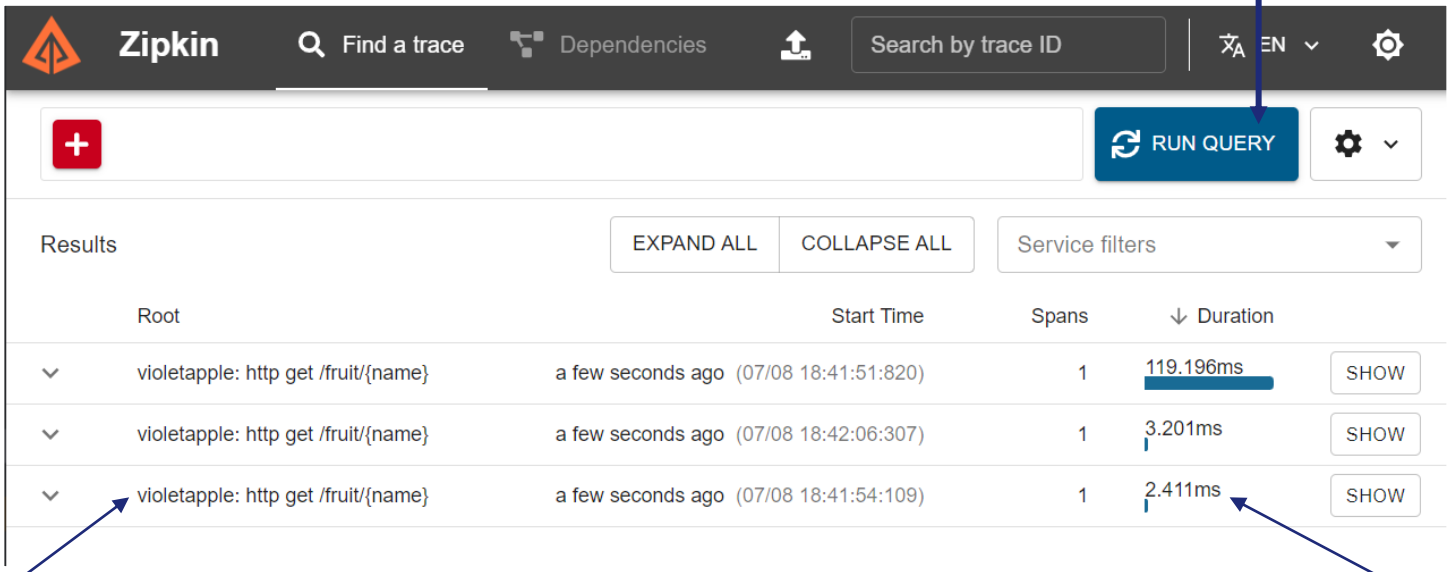
Add the following dependency to pom.xml

```
#Zepkin
management.tracing.sampling.probability=1.0
```

Add the following to application.properties

Zipkin Dashboard

After doing API calls click on run query



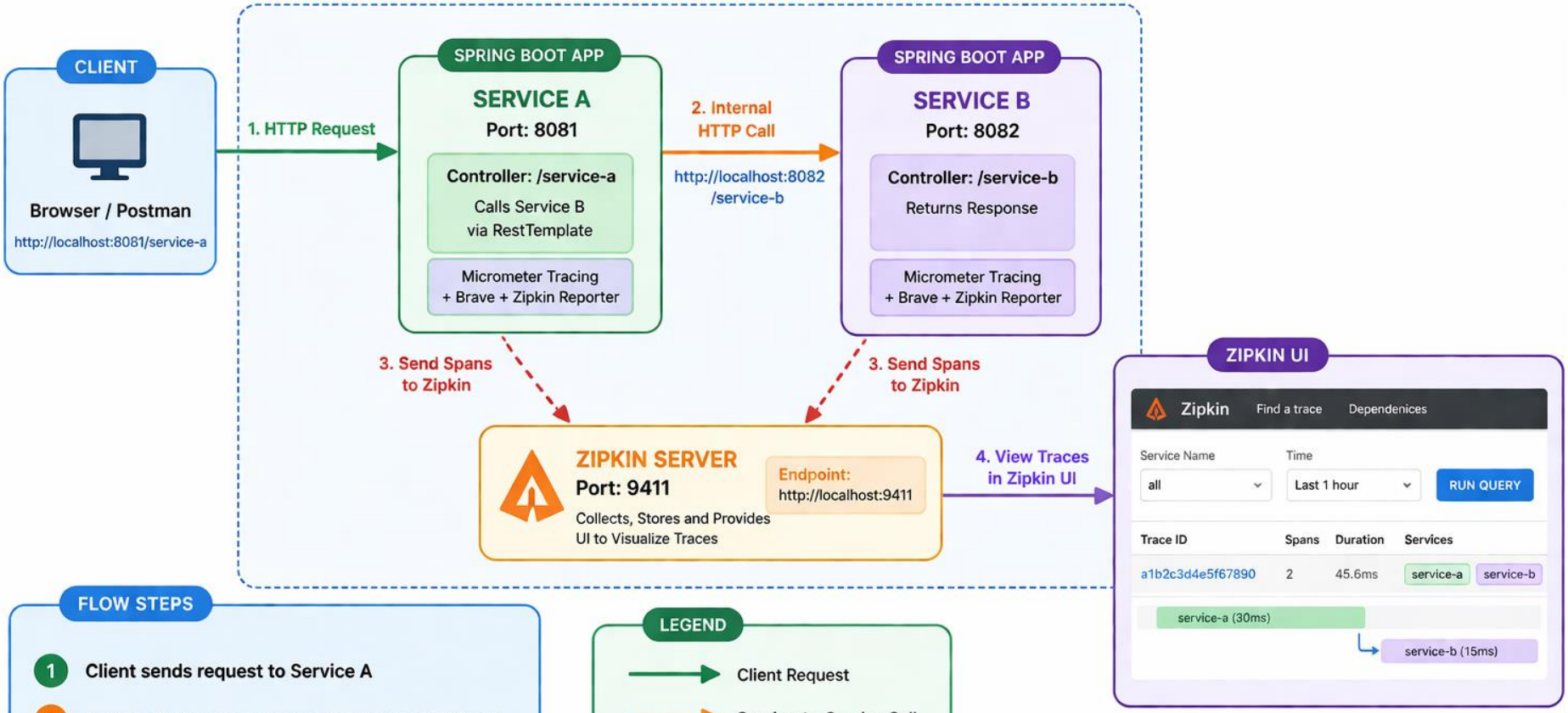
Api calls

URL for accessing Zipkin server:
http://127.0.0.1:9411/zipkin/

Duration of api call

SPRING BOOT + ZIPKIN: DISTRIBUTED TRACING DEMO

Service A Calls Service B - Traces Collected and Visualized in Zipkin



FLOW STEPS

1. Client sends request to Service A
2. Service A processes request and calls Service B
3. Both services send their tracing spans to Zipkin
4. Zipkin UI displays the collected trace showing Service A -> Service B flow

LEGEND

- Client Request
- Service-to-Service Call (Traced)
- Send Spans to Zipkin
- Spring Boot Services with Tracing Enabled

WHAT YOU WILL SEE IN ZIPKIN:

- Same Trace ID for both Service A and Service B
- Service A as Parent Span
- Service B as Child Span
- Timeline showing time spent in each service

MICROSERVICES WITH EUREKA + GATEWAY + ZIPKIN

Product + Billing + User Example with Distributed Tracing

CLIENT

Browser / Postman

http://localhost:8080/biling/1/101

1 Sends Request to Gateway

1 Sends Reqrues to Gateway

SPRING BOOT APP

API GATEWAY

Port: 8080

API GATEWAY

Port: 8080

- Service Registry
- http://localhost:8761

2 Routes Requests

PRODUCT SERVICE

PRODUCT 6082

Port: 8082

/product/{id}

USER SERVICE

USER 6081

Port: 8081

/user/{id}

Servicing Discovery

Service Discovery

3 Calls

BILLING SERVICE

BILLING 6083

Port: 8083

Calls Product + User Service

/billing/{userId}/{productId}

SPRING CLOUD NETFLIX EUREKA

EUREKA SERVER

Port: 8761

Service Registry

Endpoint: http://localhost:8761

4 Sends Traces to Zipkin

4 Visualizes Trace

FLOW STEPS

1. Client sends request to Gateway
2. Gateway sends requests to respective Services
3. Services call each other as needed, sending traces to Zipkin
4. Zipkin UI visualizes the Trace showing Service A – B → C flow

Zipkin | Find a trace | Dependencies

Service Name | Stans | Last 1 hour | RUN QUERY

Trace ID	Spans	Duration	Services
a1B2c3d4e567890		276ms	gateway billing

spteway | billing | portice | user

iproduct | ucea time

WHAT YOU WILL LEARN / OBSERVE:

- Central AP1 Gateway routing to Microervtes
- Service Discovery with Eurcka
- Distributee Tracing with Zipkin

LEGEND

- Client Request
- Routing to Services
- Send Traces to Zipkin

Summary

Concept	**Meaning**
Trace	Full request journey
Span	One unit of work
Trace ID	Unique request ID
Span ID	Unique operation ID

Key Analysis Features in Zipkin UI

- Readings from a Zipkin server are analyzed primarily through its built-in Web UI, which provides visual tools to troubleshoot latency, analyze service dependencies, and debug errors in distributed systems.
- The analysis centers on **traces** (the entire journey of a request) and **spans** (individual units of work, such as an HTTP call or database query).
- **Trace View (Timeline Analysis):** When a specific trace is selected, Zipkin displays a hierarchical timeline of all spans involved in a request. This allows engineers to visualize:
  - ✓ **Latency Bottlenecks:** Identify which service or database call is taking the longest time.
  - ✓ **Service Interactions:** See how a request hops from service A to service B to service C.
  - ✓ **Span Details:** Clicking on a span reveals annotations and tags, such as HTTP status codes, method names, or error messages, aiding in debugging.

contd..

- **Search and Filter:** Users can search for traces based on criteria like service name, operation name, tags, and duration, allowing them to isolate slow or failing requests.
- **Dependency Diagram:** Zipkin automatically generates a visual diagram showing how services are connected based on the recorded traces. This is crucial for understanding system architecture, identifying bottlenecks, and spotting excessive calls to deprecated services.
- **Percentage Time/Latency Metrics:** Zipkin highlights the percentage of time spent in each service, allowing engineers to pinpoint where the majority of latency occurs.

Analysis Workflow

- **Instrumentation:** Applications are instrumented (e.g., using Brave, Spring Cloud Sleuth, or OpenTelemetry) to send data to the Zipkin collector.
- **Collection & Storage:** The collector validates data and stores it in backends like Cassandra, Elasticsearch, or MySQL.
- **Visualization:** Engineers open the Zipkin UI (http://localhost:9411 by default) to query these stored traces.
- **Debugging/Optimization:** By examining the dependency graphs and span timelines, engineers identify overutilized services or misconfigured network calls.

3. Advanced Analysis Techniques

- **Log Correlation:** If Trace IDs are included in application logs, engineers can jump directly from a log entry in a logging system (like ELK) to the corresponding trace in Zipkin for detailed context.
- **ELK Stack Integration:** For long-term storage and advanced analytics, Zipkin is often used with Elasticsearch and Kibana, allowing for complex queries and dashboards beyond the default UI.
- **Sampling:** To manage performance impact and data volume, Zipkin uses sampling to analyze a subset of traces, which still provides enough data for troubleshooting performance bottlenecks.

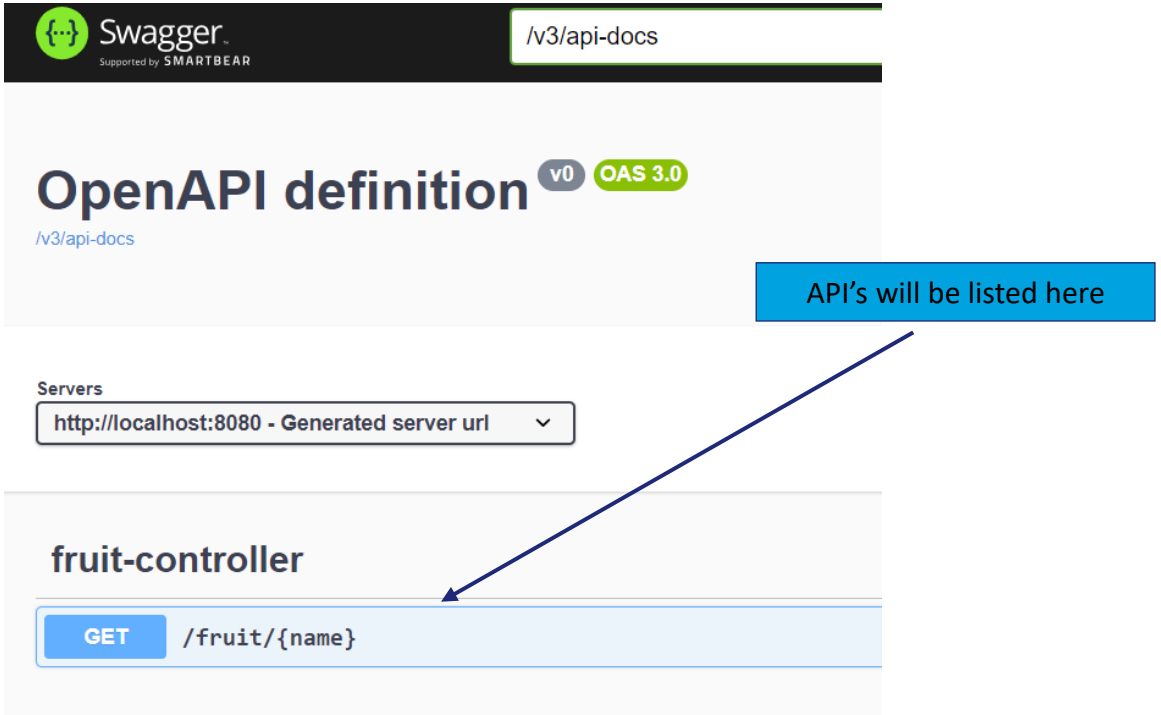
Introduction to Swagger

1. An open-source framework for API development
2. Provides a standardized way to describe, produce, consume, and visualize RESTful web services
3. The Swagger ecosystem includes several tools: Swagger UI, Swagger Editor, Swagger Codegen

Setting Up Swagger

Add the dependency to pom.xml

```
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>2.6.0</version>
</dependency>
```

The view

Url for swagger: http://localhost:8080/swagger-ui/index.html